

Tradução Dirigida por Sintaxe

Thierson Couto Rosa

Instituto de Informática
Universidade Federal de Goiás

18 de março de 2026

- É possível associar **ações** à produções de uma gramática para que alguma **computação** seja feita à medida que a análise sintática envolvendo as produções da gramática ocorra.
- Essa computação pode servir para diferentes propósitos. Por exemplo, seja: $G = (V_n = \{E, T, F\}, V_T = \{(\,, \,), +, *digit\}, P = \{L \rightarrow E, E \rightarrow E + T, E \rightarrow T, T \rightarrow T * F, T \rightarrow F, F \rightarrow (E), F \rightarrow digit\})$. Podemos usar a G que descreve expressões aritméticas na ordem infixada para: a) verificar se na entrada ocorre uma expressão correta e computar o seu valor (interpretação), ou b) traduzir para outra expressão em ordem pré-fixada (compilação), ou c) traduzir para outra expressão em ordem pós-fixada (compilação).
- As três computações acima, podem ser obtidas, usando a mesma gramática G , mas associando ações apropriadas a cada produção de G para que o objetivo final ocorra.

- Existem formalismos que permitem associar ações (computações) a cada produção de uma gramática, de modo que à medida que análise sintática ocorre, as computações são executadas. Essa associação entre análise sintática e computação é denominada *tradução dirigida por sintaxe*.
- Apesar do termo “tradução”, a associação de regras semânticas tem uma ampla aplicação, por exemplo, para analisar se a semântica de um programa está correta, ou para gerar um valor ao final da análise.
- Na tradução dirigida por sintaxe, geralmente identifica-se **atributos** ou propriedades de símbolos de uma GLC que precisem ser computados e escreve-se **equações de atributos** ou **regras semânticas** que expressem a computação desses atributos.
- Assim, a computação dos atributos fica associada às produções e símbolos (terminais e não terminais) da GLC.

Mecanismos de Tradução Dirigida por sintaxe.

- Dois formalismos são geralmente utilizados para representar uma tradução dirigida por sintaxe:
 - b) *Esquema de tradução*
 - a) *Gramática de atributos* ou *definição dirigida por sintaxe*
- Na primeira etapa da disciplina estudaremos apenas os esquemas de tradução, pois alguns deles podem ser usados diretamente com as ferramentas Bison e Yacc e serão utilizados para a geração da árvore sintática abstrata no trabalho T1.
- Antes porém, veremos o conceito de *atributo* que é utilizado por ambos os formalismos.

Atributo

- Um atributo é qualquer propriedade de um símbolo de uma gramática livre de contexto (BNF).
- Os atributos podem apresentar grande variação em seu conteúdo e complexidade.
- Exemplos:
 - tipo de dado de uma variável,
 - valor de uma expressão,
 - localização de uma variável na memória,
 - código objeto de uma função,
 - quantidade de dígitos
- Dado um símbolo X da gramática, nos referimos a um atributo a de X por $X.a$.

Esquema de Tradução

- Um *esquema de tradução* corresponde a uma GLC em que atributos são associados a símbolos da gramática e as ações semânticas, envolvidas entre chaves ({ e }), são inseridas nos lados direitos das produções, indicando o estágio da análise sintática em que elas devem ser executadas para computar os atributos.
- Assim, na produção $B \rightarrow X \{a\} Y$, a ação a é executada após o reconhecimento de X se X for um terminal, ou após o reconhecimento de todos os terminais derivados X (se X é não terminal), mas é executada antes de Y ser processado.
- Dada uma produção de uma GLC: $X_0 \rightarrow X_1 X_2 \cdots X_n$, o valor do atributo $X_i.a_j$ de cada símbolo X_i pode ser calculado através de uma equação de atributos ou ação semântica da forma:

$$X_i.a_j = f(X_0.a_1, \cdots X_0.a_{k_0}, X_1.a_1, \cdots X_1.a_{k_1}, \cdots X_n.a_1, \cdots X_n.a_{k_n})$$

Esquema de Tradução

- A equação acima é geral. Na prática, o cálculo de $X_i.a_j$ pode depender apenas de parte dos atributos de apenas parte dos demais símbolos da produção.
- É comum que um mesmo não-terminal apareça do lado esquerdo e direito de uma produção (recursividade). Exemplo: $E \rightarrow E + T$. Nesse casos utilizamos índices para diferenciar os atributos de ocorrências distintas do mesmo símbolo gramatical. Exemplo: $E_1.val = E_2.val + T.val$. Esse exemplo, também mostra que no caso em que a regra semântica corresponde a uma expressão com operadores aritméticos, a expressão pode ser usada diretamente no lugar de uma função que efetua o cálculo da regra semântica.

Exemplo 1

Seja a gramática simples para número sem sinal:

$$\textit{número} \rightarrow \textit{número dígito} \mid \textit{dígito}$$
$$\textit{dígito} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

Obter um esquema de tradução para computar o valor na base decimal de um número, usando a gramática acima.

Exemplo 1 - continuação

$numero_1 \rightarrow numero_2 \textit{ digito}$	$\{numero_1.val = numero_2.val * 10 + \textit{ digito.val}\}$
$numero \rightarrow \textit{ digito}$	$\{numero.val = \textit{ digito.val}\}$
$\textit{ digito} \rightarrow 0$	$\{\textit{ digito.val} = 0\}$
$\textit{ digito} \rightarrow 1$	$\{\textit{ digito.val} = 1\}$
$\textit{ digito} \rightarrow 2$	$\{\textit{ digito.val} = 2\}$
$\textit{ digito} \rightarrow 3$	$\{\textit{ digito.val} = 3\}$
$\textit{ digito} \rightarrow 4$	$\{\textit{ digito.val} = 4\}$
$\textit{ digito} \rightarrow 5$	$\{\textit{ digito.val} = 5\}$
$\textit{ digito} \rightarrow 6$	$\{\textit{ digito.val} = 6\}$
$\textit{ digito} \rightarrow 7$	$\{\textit{ digito.val} = 7\}$
$\textit{ digito} \rightarrow 8$	$\{\textit{ digito.val} = 8\}$
$\textit{ digito} \rightarrow 9$	$\{\textit{ digito.val} = 9\}$

Exemplo 1 - observações

- Repare no exemplo anterior que cada dígito (os terminais, $0, 1, \dots, 9$) tem um atributo numérico *val* que corresponde ao caractere que ele representa.
- Enquanto 0 na produção *dígito* $\rightarrow 0$ representa o caractere '0', o 0 em *digito.val* = 0 representa o número inteiro 0.
- Nesse exemplo, cada produção tem apenas uma regra semântica que aparece após o último símbolo do lado direito da produção. Há casos em que várias regras podem estar associadas a uma mesma produção. Elas podem aparecer entre '{' e '}' em diferentes posições do lado direito de uma produção.
- Também, nesse esquema de tradução de atributos há apenas um tipo de atributo (*val*). Poderia haver mais.

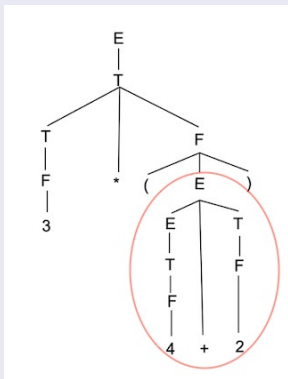
Exemplo 2

Escreva um esquema de tradução para implementar uma calculadora simples que utiliza os operadores de adição, subtração e multiplicação. Considere a gramática abaixo: $E \rightarrow E + T$, $E \rightarrow T$, $T \rightarrow T * F$, $T \rightarrow F$, $F \rightarrow (E)$, $F \rightarrow num$.

- Esboço da solução:
 - Gerar uma árvore de derivação para uma cadeia válida ajuda no projeto do esquema de tradução.
 - Com a árvore de derivação fica mais fácil visualizar o caminho que os dados precisam percorrer na árvore para que os cálculos dos atributos sejam feitos corretamente.
 - A árvore nos permite ver também a relação entre os atributos e em que posição do lado direito de cada produção precisamos inserir a ação semântica que irá computar cada atributo.

Exemplo2

Árvore de derivação para a cadeia:
 $3 * (4 + 2)$

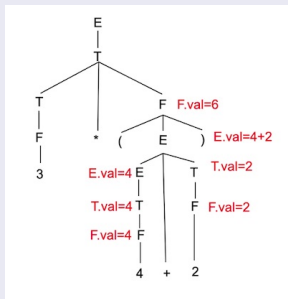


Esquema de tradução

- Repare que a expressão E entre parênteses corresponde a $E_1 \rightarrow E_2 + T$. Precisamos propagar o valor 4 até E_2 e o valor 2 até T .
- A propagação pode ser feita através de atributos de F , E e T computados em ações semânticas para as seguintes produções: $F \rightarrow num$, $T \rightarrow F$, $E \rightarrow T$.

Exemplo2

Árvore de derivação para a cadeia:
 $3 * (4 + 2)$

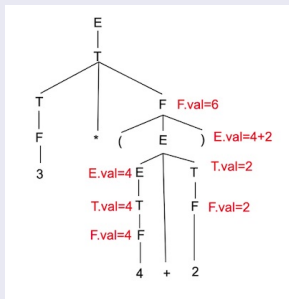


Esquema de tradução (parcial)

- Conseguimos as propagações dos valores, associando ações semânticas às produções correspondentes:
 $F \rightarrow num \{F.val = atoi(num); \}$

Exemplo2

Árvore de derivação para a cadeia:
 $3 * (4 + 2)$

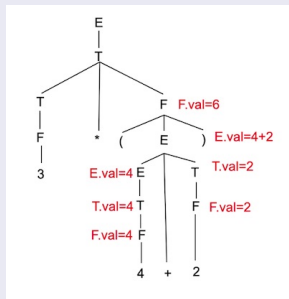


Esquema de tradução (parcial)

- Conseguimos as propagações dos valores, associando ações semânticas às produções correspondentes:
 $F \rightarrow \text{num} \{F.val = \text{atoi}(\text{num});\}$
 $F \rightarrow (E) \{F.val = E.val;\}$
 $T \rightarrow F \{T.val = F.val;\}$

Exemplo2

Árvore de derivação para a cadeia:
 $3 * (4 + 2)$



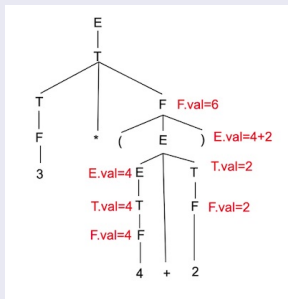
Esquema de tradução (parcial)

- Conseguimos as propagações dos valores, associando ações semânticas às produções correspondentes:
 $F \rightarrow num \{F.val = atoi(num);\}$
 $F \rightarrow (E) \{F.val = E.val;\}$
 $T \rightarrow F \{T.val = F.val;\}$
 $E \rightarrow T \{E.val = T.val;\}$

Exemplo2

Árvore de derivação para a cadeia:

$3 * (4 + 2)$



Esquema de tradução (parcial)

- Conseguimos as propagações dos valores, associando ações semânticas às produções correspondentes:

$F \rightarrow num \{F.val = atoi(num);\}$

$F \rightarrow (E) \{F.val = E.val;\}$

$T \rightarrow F \{T.val = F.val;\}$

$E \rightarrow T \{E.val = T.val;\}$

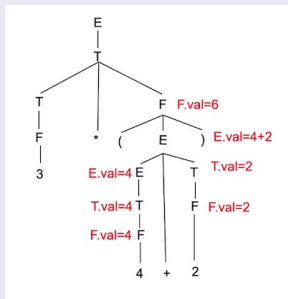
$E_1 \rightarrow E_2 + T$

$\{E_1.val = E_2.val + T.val;\}$

Exemplo2

Árvore de derivação para a cadeia:

$3 * (4 + 2)$



Esquema de tradução (parcial)

- Conseguimos as propagações dos valores, associando ações semânticas às produções correspondentes:

$F \rightarrow num \{F.val = atoi(num);\}$

$F \rightarrow (E) \{F.val = E.val;\}$

$T \rightarrow F \{T.val = F.val;\}$

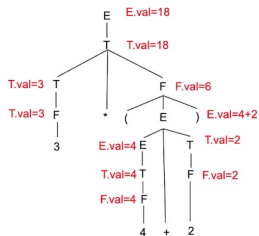
$E \rightarrow T \{E.val = T.val;\}$

$E_1 \rightarrow E_2 + T$

$\{E_1.val = E_2.val + T.val;\}$

Exemplo2

Árvore de derivação para a cadeia:
 $3 * (4 + 2)$

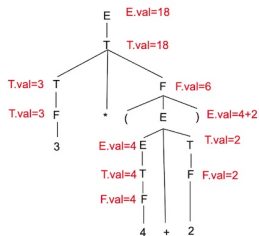


Esquema de tradução (completo)

$$E_1 \rightarrow E_2 + T \{E_1.val = E_2.val + T.val\}$$
$$E \rightarrow T \{E.val = T.val\}$$
$$T_1 \rightarrow T_2 * F \{T_1.val = T_2.val * F.val\}$$
$$T \rightarrow F \{T.val = F.val;\}$$
$$F \rightarrow num \{F.val = atoi(num);\}$$
$$F \rightarrow (E) \{F.val = E.val\}$$

Exemplo2

Árvore de derivação para a cadeia:
 $3 * (4 + 2)$



Esquema de tradução (completo)

$$E_1 \rightarrow E_2 + T \{E_1.val = E_2.val + T.val\}$$
$$E \rightarrow T \{E.val = T.val\}$$
$$T_1 \rightarrow T_2 * F \{T_1.val = T_2.val * F.val\}$$
$$T \rightarrow F \{T.val = F.val;\}$$
$$F \rightarrow num \{F.val = atoi(num);\}$$
$$F \rightarrow (E) \{F.val = E.val\}$$

Dependência entre atributos

O esquema de tradução acima impõe uma dependência entre atributos:

- O atributo $F.val$ depende do lexema que ocorre no token num .
- O atributo $T.val$ depende apenas do atributo $F.val$ na ação associada à produção $T \rightarrow F$, mas depende de $T_1.val$ e $F.val$ na ação semântica associada à produção $T \rightarrow T_1 * F$.

Exemplo 3

- Considere a seguinte gramática para declarações de variáveis, como na sintaxe da linguagem C:

$$\begin{aligned}D &\rightarrow T L \\T &\rightarrow int \mid float \\L &\rightarrow id, L \mid id;\end{aligned}$$

- Obter um esquema de tradução para atribuir o tipo de dados associado a uma declaração (int ou float) a cada identificador da lista de identificadores.

Exemplo 3

- Considere a seguinte gramática para declarações de variáveis, como na sintaxe da linguagem C:

$$\begin{aligned}D &\rightarrow T L \\T &\rightarrow int \mid float \\L &\rightarrow id, L \mid id;\end{aligned}$$

- Obter um esquema de tradução para atribuir o tipo de dados associado a uma declaração (int ou float) a cada identificador da lista de identificadores.
- Criamos os seguintes atributos: *id.tipo*, *T.tipo*, *L.tipo*.

Exemplo 3

- Considere a seguinte gramática para declarações de variáveis, como na sintaxe da linguagem C:

$$\begin{aligned}D &\rightarrow T L \\T &\rightarrow int \mid float \\L &\rightarrow id, L \mid id;\end{aligned}$$

- Obter um esquema de tradução para atribuir o tipo de dados associado a uma declaração (int ou float) a cada identificador da lista de identificadores.
- Criamos os seguintes atributos: *id.tipo*, *T.tipo*, *L.tipo*.
- O esquema de tradução tem como objetivo **computar os valores de *id.tipo*** para cada elemento da lista de identificadores.

Exemplo 3

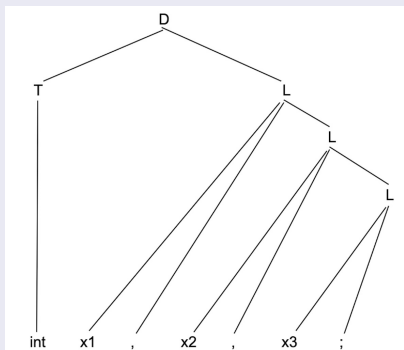
- Considere a seguinte gramática para declarações de variáveis, como na sintaxe da linguagem C:

$$\begin{aligned}D &\rightarrow T L \\T &\rightarrow \textit{int} \mid \textit{float} \\L &\rightarrow \textit{id}, L \mid \textit{id};\end{aligned}$$

- Obter um esquema de tradução para atribuir o tipo de dados associado a uma declaração (*int* ou *float*) a cada identificador da lista de identificadores.
- Criamos os seguintes atributos: *id.tipo*, *T.tipo*, *L.tipo*.
- O esquema de tradução tem como objetivo **computar os valores de *id.tipo*** para cada elemento da lista de identificadores.
- Para nos ajudar, novamente utilizaremos uma árvore de derivação para uma cadeia de entrada: `int x1, x2, x3;`

Exemplo 3

Árvore de derivação para a cadeia:
int x1, x2, x3;

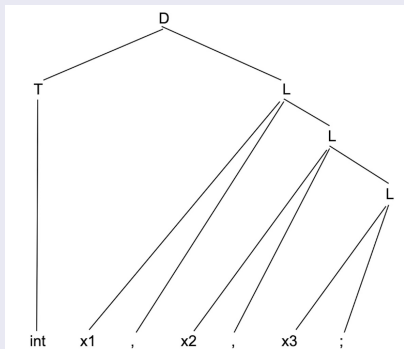


Esquema de tradução (esboço)

- Precisamos armazenar o tipo da declaração no atributo (*tipo*) de *T*:

Exemplo 3

Árvore de derivação para a cadeia:
int x1, x2, x3;

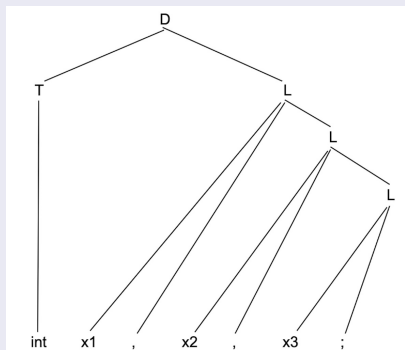


Esquema de tradução (esboço)

- Precisamos armazenar o tipo da declaração no atributo (*tipo*) de *T*:
- $T \rightarrow int\{T.tipo = INT\};$
 $T \rightarrow float\{T.tipo = FLOAT\}$

Exemplo 3

Árvore de derivação para a cadeia:
int x1, x2, x3;

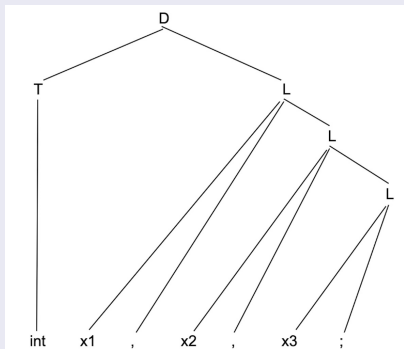


Esquema de tradução (esboço)

- Precisamos armazenar o tipo da declaração no atributo (*tipo*) de *T*:
- $T \rightarrow int\{T.tipo = INT\};$
 $T \rightarrow float\{T.tipo = FLOAT\}$
- *L.tipo* (sub-topo da árvore) precisa receber o tipo:
- $D \rightarrow T\{L.tipo = T.tipo\}L$
- Precisamos propagar a informação de tipo em *L* para baixo na árvore:

Exemplo 3

Árvore de derivação para a cadeia:
int x1, x2, x3;

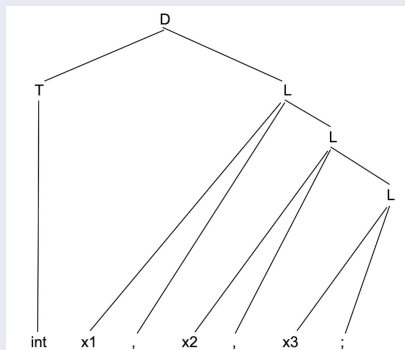


Esquema de tradução (esboço)

- Precisamos armazenar o tipo da declaração no atributo (*tipo*) de *T*:
- $T \rightarrow int\{T.tipo = INT\};$
 $T \rightarrow float\{T.tipo = FLOAT\}$
- *L.tipo* (sub-topo da árvore) precisa receber o tipo:
- $D \rightarrow T\{L.tipo = T.tipo\}L$
- Precisamos propagar a informação de tipo em *L* para baixo na árvore:
- $L_1 \rightarrow id\{id.tipo = L_1.tipo\}, \{L_2.tipo = L_1.tipo\}L_2$

Exemplo 3

Árvore de derivação para a cadeia:
int x1, x2, x3;

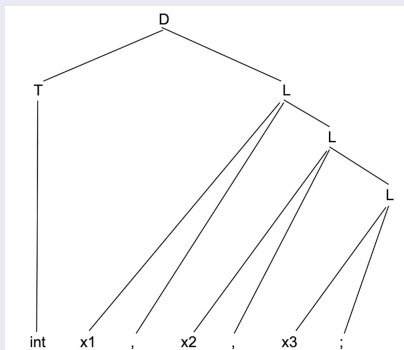


Esquema de tradução (esboço)

- Precisamos armazenar o tipo da declaração no atributo (*tipo*) de *T*:
- $T \rightarrow int\{T.tipo = INT\};$
 $T \rightarrow float\{T.tipo = FLOAT\}$
- *L.tipo* (sub-topo da árvore) precisa receber o tipo:
- $D \rightarrow T\{L.tipo = T.tipo\}L$
- Precisamos propagar a informação de tipo em *L* para baixo na árvore:
- $L_1 \rightarrow id\{id.tipo = L_1.tipo\}, \{L_2.tipo = L_1.tipo\}L_2$
- $L \rightarrow id\{id.tipo = L.tipo\};$

Exemplo 3

Árvore de derivação para a cadeia:
int x1, x2, x3;



Esquema de tradução (esboço)

- Precisamos armazenar o tipo da declaração no atributo (*tipo*) de T :
- $T \rightarrow int\{T.tipo = INT\};$
 $T \rightarrow float\{T.tipo = FLOAT\}$
- $L.tipo$ (sub-topo da árvore) precisa receber o tipo:
- $D \rightarrow T\{L.tipo = T.tipo\}L$
- Precisamos propagar a informação de tipo em L para baixo na árvore:
- $L_1 \rightarrow id\{id.tipo = L_1.tipo\}, \{L_2.tipo = L_1.tipo\}L_2$
- $L \rightarrow id\{id.tipo = L.tipo\};$

Diferença do fluxo de informação nos dois exemplos

- No Exemplo 2:
 - Os valores dos números propagam sempre para cima na árvore de derivação.
 - Os atributos dos nós pais (não-terminais) **dependem** dos atributos de seus símbolos filhos na árvore.
 - O objetivo é computar o valor da expressão, que corresponde ao atributo *val* do símbolo na raiz a árvore.
- No Exemplo 3:
 - O valor do tipo precisa inicialmente subir na árvore até o atributo *tipo* de *T*.
 - Em seguida, o valor do tipo da declaração precisa ser propagado para o irmão à direita de *T* que é a lista de identificadoros.
 - A partir desse ponto, o valor de tipo deve propagar para os filhos de *L* em cada ocorrência recursiva de *L* na árvore derivação.

Dependência entre Atributos

- Nos dois últimos exemplos fica claro que as regras semânticas estabelecem dependências entre atributos. Por exemplo, no esquema de tradução do exemplo 3, o atributo $L.tipo$ na produção $D \rightarrow TL$, depende do atributo $T.tipo$. **Nesse caso, o atributo $L.tipo$ depende do atributo de um símbolo irmão à esquerda na produção.**
- Já no seguinte trecho do esquema de tradução:
 $L_1 \rightarrow id\{id.tipo = L_1.tipo\}, \{L_2.tipo = L_1.tipo\}L_2$, **os atributos $id.tipo$ e L_2 dependem do atributo $tipo$ de L_1 , ou seja, de um atributo do nó pai deles na produção.**
- Porém, nos trechos: $T \rightarrow int\{T.tipo = INT\}$ e $T \rightarrow float\{T.tipo = FLOAT\}$, **o atributo $T.tipo$ do nó pai T depende do token na entrada (INT ou FLOAT), ou seja, depende de um atributo de um nó filho.**

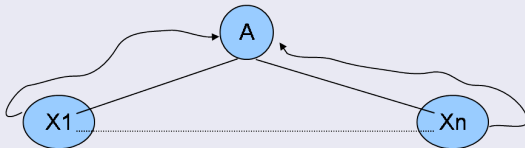
Tipos de Atributos

Os atributos são classificados de acordo com suas dependência de outros atributos em dois tipos:

- Atributos sintetizados.
- Atributos herdados.

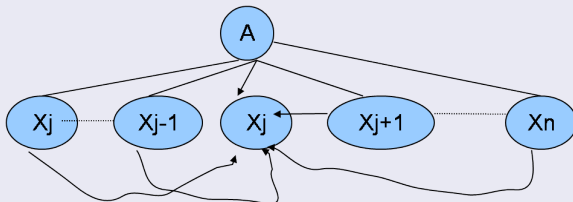
Atributos Sintetizados

Dada uma produção: $A \rightarrow X_1 \dots X_n$. Um atributo a de A é sintetizado se sua computação depende apenas de atributos de símbolos do lado direito da produção, ou seja, de $X_1, \dots, X_n$.



Atributos herdados

- Dada uma produção: $A \rightarrow X_1 \cdots X_j \cdots X_n$. Um atributo a de X_j é herdado se sua computação depende de atributos de A ou de atributos de símbolos $X_1, \cdots X_{j-1}$ ou $X_{j+1}, \cdots X_n$, ou ainda, de atributos de símbolos do lado direito de X_j -produções.



Atributos dos exemplos 1, 2 e 3

- Indique qual o tipo dos seguintes atributos do exemplo 1:
digito.val, *numero.val*.

Atributos dos exemplos 1, 2 e 3

- Indique qual o tipo dos seguintes atributos do exemplo 1:
digito.val, *numero.val*.
- Ambos são sintetizados pois dependem exclusivamente de atributos de nós filhos.

Atributos dos exemplos 1, 2 e 3

- Indique qual o tipo dos seguintes atributos do exemplo 1:
digito.val, *numero.val*.
- Ambos são sintetizados pois dependem exclusivamente de atributos de nós filhos.
- Indique qual o tipo dos seguintes atributos do exemplo 2:
F.val, *T.val* e *E.val*.

Atributos dos exemplos 1, 2 e 3

- Indique qual o tipo dos seguintes atributos do exemplo 1:
digito.val, *numero.val*.
- Ambos são sintetizados pois dependem exclusivamente de atributos de nós filhos.
- Indique qual o tipo dos seguintes atributos do exemplo 2:
F.val, *T.val* e *E.val*.
- Todos são sintetizados pois dependem exclusivamente de atributos de nós filhos.

Atributos dos exemplos 1, 2 e 3

- Indique qual o tipo dos seguintes atributos do exemplo 1:
digito.val, *numero.val*.
- Ambos são sintetizados pois dependem exclusivamente de atributos de nós filhos.
- Indique qual o tipo dos seguintes atributos do exemplo 2:
F.val, *T.val* e *E.val*.
- Todos são sintetizados pois dependem exclusivamente de atributos de nós filhos.
- Indique qual o tipo dos seguintes atributos do exemplo 3:
T.tipo, *id.tipo* e *L.tipo*.

Atributos dos exemplos 1, 2 e 3

- Indique qual o tipo dos seguintes atributos do exemplo 1:
digito.val, *numero.val*.
- Ambos são sintetizados pois dependem exclusivamente de atributos de nós filhos.
- Indique qual o tipo dos seguintes atributos do exemplo 2:
F.val, *T.val* e *E.val*.
- Todos são sintetizados pois dependem exclusivamente de atributos de nós filhos.
- Indique qual o tipo dos seguintes atributos do exemplo 3:
T.tipo, *id.tipo* e *L.tipo*.
- *T.tipo* é sintetizado, *id.tipo* e *L.tipo* são herdados. No caso de *L.tipo*, ele é herdado ora de um irmão (T), ora de um atributo de seu nó pai, dependendo da ação semântica considerada.

Tipos de análise sintática

- Existem basicamente dois tipos de análise sintática: análise *sintática descendente* (*top-down*) e análise *sintática ascendente* (*bottom-up*).
- Na análise sintática descendente, o *parser* simula uma derivação mais à esquerda da cadeia de entrada.
- Isso equivale a construir uma árvore de derivação de cima para baixo e da esquerda para direita.
- O símbolo "pai" surge primeiro que os filhos. Os "irmãos" mais à esquerda surgem primeiro que os mais à direita na árvore de derivação.

Tipos de análise sintática

- Na análise ascendente, o parser simula o reverso de uma derivação mais à direita. Esse reverso de derivação é denominado **redução**. O lado direito de uma produção é reduzido ao símbolo do lado esquerdo.
- Em outras palavras: Um nó "pai" na árvore de derivação "nasce" depois de seus filhos!
- Os não-terminais de um lado direito de uma produção surgem da esquerda para direita. Isto é, seja a produção $A \rightarrow BCD$, a subcadeia da entrada derivada de B é reduzida a B antes da redução da subcadeia seguinte ser reduzida a C . Analogamente, a redução de C ocorre antes que a redução de D ocorra. Finalmente, o lado direito BCD é reduzido a A .

Conflitos entre ordem das ações semânticas e análise sintática

- A ordem com que a análise sintática ocorre pode tornar complicada a execução de um esquema de tradução.
- O exemplo 3, a rigor, não é possível de ser executado durante uma análise sintática ascendente.
- Isso ocorre porque a dependência de atributos entre os símbolos não combina com a ordem em que eles surgem na análise ascendente.
- O atributo $L.tipo$ em $D \rightarrow TL$ precisa ser copiado de $T.tipo$ para que as subárvores com raiz L propaguem a informação de tipo para baixo nas suas subárvores, mas as subárvores surgem de baixo para cima na análise ascendente.

Conflitos entre ordem das ações semânticas e análise sintática

- Se o esquema de tradução tem apenas atributos sintetizados, quais tipos de análise sintática facilitam sua execução?

Conflitos entre ordem das ações semânticas e análise sintática

- Se o esquema de tradução tem apenas atributos sintetizados, quais tipos de análise sintática facilitam sua execução?
- Resposta: funciona bem tanto com a análise sintática ascendente quanto com a descendente.
- Se o esquema de tradução possui alguns atributos herdados, quais tipos de análise sintática facilitam sua execução?

Conflitos entre ordem das ações semânticas e análise sintática

- Se o esquema de tradução tem apenas atributos sintetizados, quais tipos de análise sintática facilitam sua execução?
- Resposta: funciona bem tanto com a análise sintática ascendente quanto com a descendente.
- Se o esquema de tradução possui alguns atributos herdados, quais tipos de análise sintática facilitam sua execução?
- Resposta: análise ascendente não facilita. Em alguns casos inviabiliza. A análise descendente funciona bem, se os atributos herdados dependem sempre apenas de atributos do não-terminal à esquerda da produção ou de atributos de símbolos que estão à sua esquerda no lado direito de uma produção. Se o atributo herdado depende de um atributo de um símbolo à direita, nenhum dos dois tipos de análise sintática é apropriado.

Esquema de tradução para gerar uma representação Intermediária

- Como fazer quando ocorre esse conflito entre a ordem da análise e a ordem necessária para a computação os atributos?

Esquema de tradução para gerar uma representação Intermediária

- Como fazer quando ocorre esse conflito entre a ordem da análise e a ordem necessária para a computação os atributos?
- Uma solução muito utilizada consiste em postergar o cálculo dos atributos para depois da análise sintática. Nesse caso, utiliza-se um esquema de tradução apenas para gerar **uma estrutura de dados que contenha todos os atributos a serem calculados**.
- Uma estrutura **fácil** de ser criada é a **própria árvore de derivação da cadeia de entrada**.
- Uma vez criada a árvore de derivação pode-se fazer caminhamentos na mesma, de acordo com a ordem de dependência dos atributos associados aos nós (símbolos) da árvore.

Problemas com a solução anterior

- As ferramentas que implementam esquemas de tradução (YACC, Bison, etc), não geram automaticamente uma árvore de derivação. Portanto, seria necessário primeiro construir a árvore de derivação em memória usando um esquema de tradução anterior.
- No caso de um compilador, a árvore de derivação gerada seria uma representação intermediária do programa fonte. A análise semântica poderia se dar através de caminhamentos na árvore de derivação, de modo independente da análise sintática.
- O problema é que uma árvore de derivação é geralmente uma estrutura muito complexa para representar um programa fonte: nós para não terminais e altura desnecessariamente longa da árvore devido ao excesso de indireções do tipo: $A \Rightarrow B \Rightarrow \dots$

Árvore Sintática Abstrata

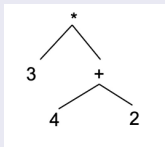
- Quando uma compilação em mais de duas fases é possível, é comum a geração (mesmo que parcial) de uma árvore mais simples que a árvore de derivação. Essa árvore é denominada *árvore (de sintaxe) abstrata* (ou *abstract syntax tree* - AST), por gerar uma abstração ainda maior do que uma árvore de derivação, mantendo apenas os componentes da linguagem suficientes para a análise semântica/geração de código.
- Os símbolos não-termais da linguagem não aparecem na árvore abstrata. Apenas as construções da linguagem que têm significado semântico são mantidas.

Árvore de sintaxe Abstrata

- A árvore abstrata é uma árvore de operadores e parâmetros e sub-expressões. Cada nó interno corresponde a um operador e os filhos desse nó correspondem a operandos que por sua vez podem ser outras sub-árvore de operadores, ou um operando terminal.
- Por exemplo, se em uma linguagem o comando *while* é definido como *while (expr) stmt*, a árvore abstrata correspondente tem um operador *while* e dois filhos. O filho à esquerda é a subárvore de operadores correspondente a *expr* e o filho da direita é a subárvore de operadores correspondente a *stmt*

Árvore Sintática Abstrata - Exemplo

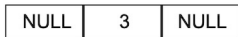
- Uma árvore abstrata para a cadeia $3 * (4 + 2)$ segundo a gramática do exemplo 2 seria:



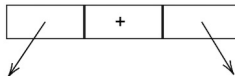
- Esceva um esquema de tradução para a gramática do G do exemplo 2, transcrita abaixo, para gerar ASTs para expressões aritméticas.
 $G = (V_n = \{E, T, F\}, V_T = \{(\,, \,), +, *digit\}, P = \{L \rightarrow E, E \rightarrow E + T, E \rightarrow T, T \rightarrow T * F, T \rightarrow F, F \rightarrow (E), F \rightarrow NUM\})$.

Árvore Sintática Abstrata

- A árvore abstrata deve ser representada na memória do computador. Logo, corresponde a uma estrutura de dados do tipo árvore. Como os operadores são binários, criaremos uma estrutura para cada nó da árvore, sendo formada por três campos:
 $\uparrow$ subárvore_esquerda, operador/num, $\uparrow$ subárvore_direita.
- A figura abaixo ilustra nós folhas e nós internos na AST para expressões aritméticas:



Exemplo de nó folha

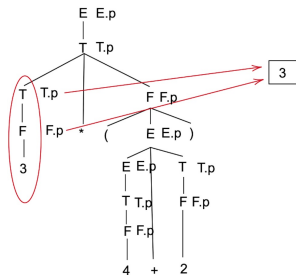


Exemplo de nó interno

- Supomos que exista uma função construtora
 $\uparrow$ no cria_no($\uparrow$ no, string, $\uparrow$ no)

Árvore de derivação

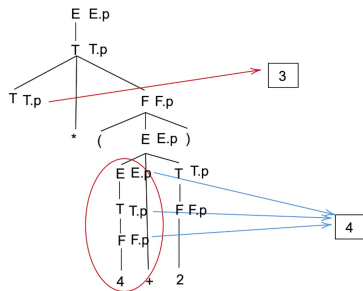
- Árvore de derivação para uma cadeia para orientar a solução.
- As elipses em vermelho mostram as sequencias do *parsing* entre um slide e outro.



Passos

- Na análise sintática ascendente, 3 é inicialmente reduzido a F . Nesse momento, precisamos criar um nó-folha para 3 e armazenar seu endereço em um atributo $F.p$ de F , pois o endereço do nó precisa ser propagado até T . Conseguimos esse feito do seguinte modo:
$$F \rightarrow NUM \{ F.p = \text{cria_no}(\text{NULL}, \text{num.lexema}, \text{NULL}) \}$$
$$T \rightarrow F \{ T.p = F.p \}$$
- A redução elimina o lado direito (filhos) da produção reduzida.

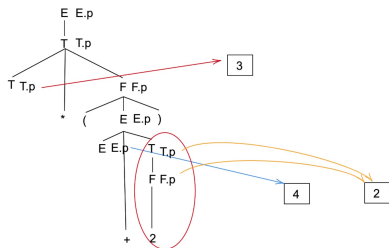
Árvore de derivação



Passos

- Em seguida, o parser lê '*', (, e lê 4.
- Nesse momento, reduz 4 a F , dispara a ação para criar um nó-folha para 4:
 $F \rightarrow NUM\{F.p = \text{cria_no}(\text{NULL}, \text{num.lexema}, \text{NULL})\}$. Em seguida, ocorrem as seguintes reduções:
 $T \rightarrow F\{T.p = F.p\}$
 $E \rightarrow T\{E.p = T.p\}$

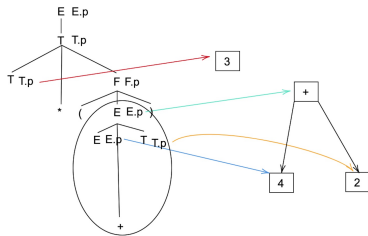
Árvore de derivação



Passos

- Em seguida, o parser lê '+', e lê 2.
- Nesse momento, reduz 2 a F , dispara a ação para criar um nó-folha para 2: $F \rightarrow NUM\{F.p = \text{cria_no}(\text{NULL}, \text{num.lexema}, \text{NULL})\}$. Em seguida, ocorre a redução. $T \rightarrow F\{T.p = F.p\}$

Árvore de derivação



Passos

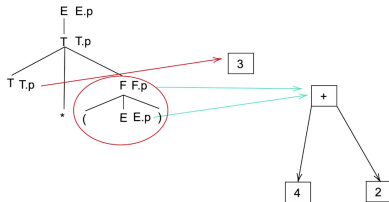
- No momento, o *parser* já reconheceu o lado direito $E_2 + T$ e precisa reduzi-lo a E_1 . Precisamos de ações semânticas para:

- criar um nó interno para "+" e copiar o seu endereço em $E_1.p$;
- fazer o primeiro filho do nó criado apontar para o endereço que está em $E_2.p$;
- fazer o segundo filho do nó criado apontar para o endereço que está em $T.p$

$$E_1 \rightarrow E_2 +$$

$T\{E_1.p = (\text{cria_no}(E_2.p, "+", T.p));\}$

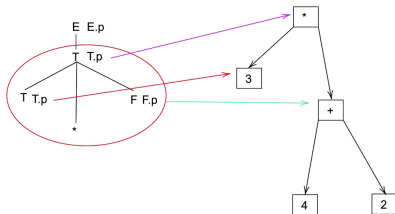
Árvore de derivação



Passos

- No momento, o *parser* se encontra em um estado em que ele já reconheceu o lado direito (E) e vai reduzi-lo a F . Nesse momento, precisamos criar uma ação semântica para copiar o endereço do nó "+" que está em $E.p$ para $F.p$: $F \rightarrow (E) \{F.p = E.p\}$

Árvore de derivação

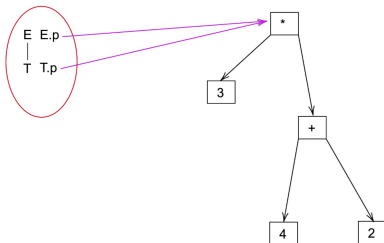


Passos

- No momento, o *parser* já reconheceu o lado direito $T_2 * F$ e precisa reduzi-lo a T_1 . Precisamos de ações semânticas para:
 - criar um nó interno para " $*$ " e copiar o seu endereço em $T_1.p$;
 - fazer o primeiro filho do nó criado apontar para o endereço que está em $T_2.p$;
 - fazer o segundo filho do nó criado apontar para o endereço que está em $F.p$

 $T_1 \rightarrow$
$$T_2 * F\{T_1.p = (\text{cria_no}(T_2.p, \text{"*"}, F.p));\}$$

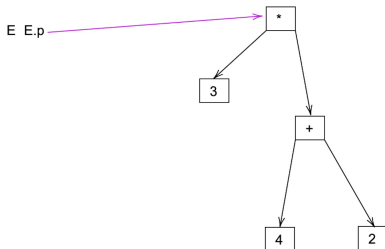
Árvore de derivação



Passos

- Falta o *parser* realizar a última redução: $E \rightarrow T$.
- Nesse momento, precisamos introduzir uma ação semântica para copiar o endereço do nó "*" que está em $T.p$ para $E.p$:
 $E \rightarrow T\{E.p = T.p\}$

Árvore de derivação



Passos

- Finalmente, temos que a cadeia de entrada foi completamente reduzida a E , símbolo inicial da gramática, e a árvore abstrata está criada na memória, sendo que o endereço da raiz da árvore abstrata está em $E.p$.

Escreva um esquema de tradução para gerar árvores sintáticas abstratas para a gramática do exemplo 3.